二维数组传参写成 int** 会怎样：两套世界模型的对撞

嵌入式 Linux 手记　2026-10-19　公众号文章 · 原链接 mp.weixin.qq.com

先猜结果。这段代码，编译器和运行时各自给什么答复？

```
void sum(int **p, int rows, int cols) {
    int s = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            s += p[i][j];
    printf("%d\n", s);
}

intmain(void) {
    int a[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    sum((int **)a, 3, 4);   /* 想累加 1~12，答案应为 78 */
    return0;
}
```

强转 `(int **)` 在场，编译器闭嘴放行。把强转拿掉再试，答案分层：gcc 一直给一条 `-Wincompatible-pointer-types` 警告——这在标准里本就是约束违反，从未被放过，只是老版本仍生成可执行文件；GCC 14 起无论 `-std` 设为哪个版本，这条警告默认升级为错误（`-fpermissive` 可降回）——编译器终于不再替错误兜底。可只要强转在场，所有版本的编译器都沉默：强转不是护身符，是替编译器闭嘴，然后由运行时付款。

篇 3 结尾用一段话预览过这个坑，落点在步长：`int (*)[4]` 每步 16 字节，`int **` 每步 8 字节。步长只是表象，根子在两套内存模型根本对不上。这一篇把模型摆上台面，讲清退化为什么不递归、哪三种签名等价、`int **` 什么时候才真的对。

一、两套世界模型：连片数据 vs 指针格

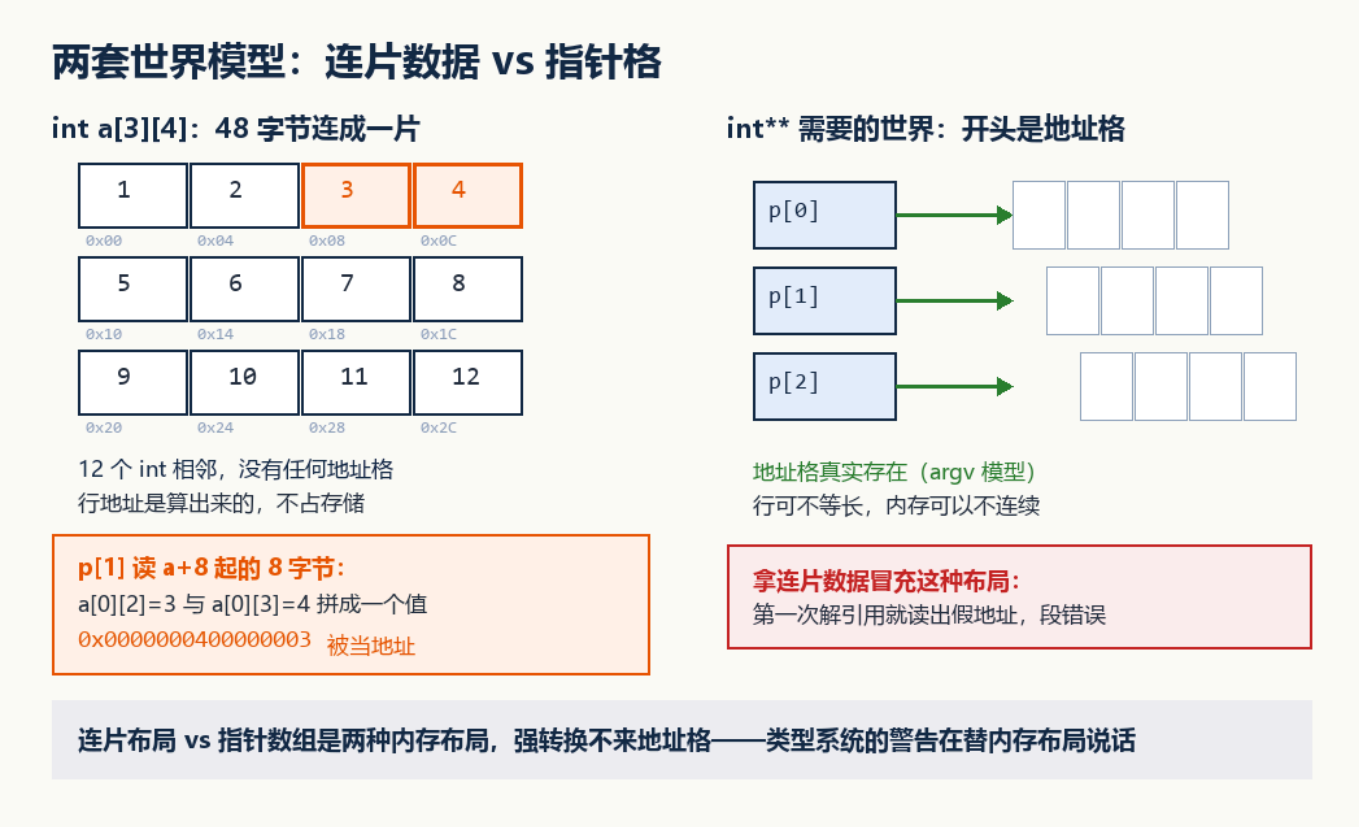
`int a[3][4]` 在内存里是一整块 48 字节（按 `int` 占 4 字节算），12 个 `int` 从低到高连成一片：

```
a[0][0] a[0][1] a[0][2] a[0][3] | a[1][0] a[1][1] ... | a[2][0] ...
0x00    0x04    0x08    0x0C    | 0x10                 | 0x20
```

这块内存里没有任何一个字节存的是"地址"。所谓"第 1 行的地址""第 0 行第 2 个元素的地址"，都只是算出来的位置，不占存储。篇 1 讲过数组退化：`a` 在表达式里退化为指向首元素的指针，值为首字节地址；但退化产物是个右值，没有存储空间，自然也不会有"指针数组"藏在哪。

`int **p` 脑补的世界完全是另一套：内存开头排着若干个 8 字节的"地址格"，每个格子里装一个地址，解一次引用 `*p` 就是读出一格，拿到的值被当成新地址再往里走。`p[i][j]` 的翻译是"从 `p` 起跳 `i` 格读出地址，再从这个地址起跳 `j` 格读数据"。

对撞发生在第一次解引用。64 位平台上 `p[1]` 读的是 `a + 8` 起的 8 个字节，也就是 `a[0][2]` 与 `a[0][3]` 两个 `int` 拼成的 8 字节值。按上面的初始化，拼出来是 `0x0000000400000003` 一类的数字，拿它当地址再解引用，MMU 直接拒绝：段错误。篇 3 预览时说过，`p[1]` 读到的是 `a[0][2]` 与 `a[0][3]` 拼成的假地址；这一篇按字节把这笔账算到底。



图：两套世界模型对照。左边 48 字节连片没有一个地址格；右边 `int**` 需要的世界开头就是地址格。拿连片冒充指针数组，第一次解引用就撞上假地址。

二、退化为什么不递归：标准只降第一维

那编译器本该坚持什么类型？C11 §6.7.6.3 第 7 段规定：形参声明为"array of type"的，调整为"pointer to type"。注意调整只发生一次、只降最外层：

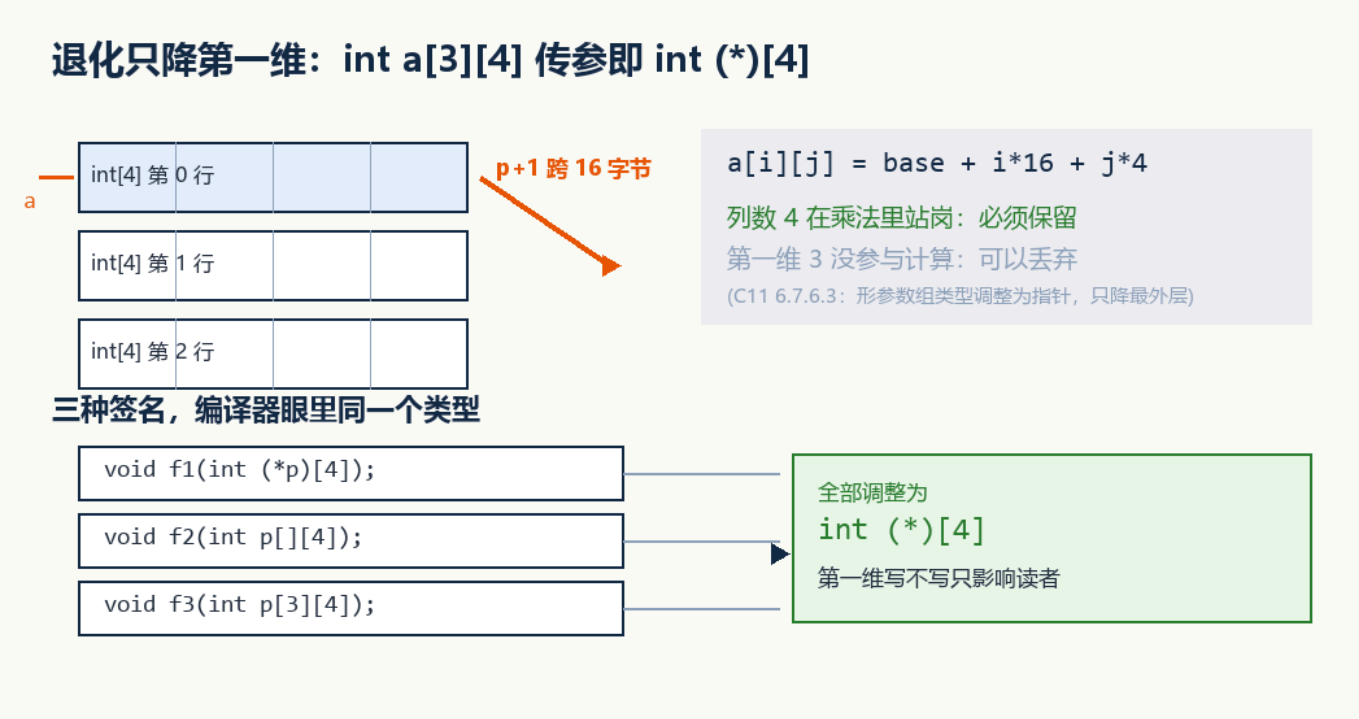
```
void f2(int a[3][4]);    /* 调整为 int (*a)[4] */
void f3(int a[3][4][5]); /* 调整为 int (*a)[4][5] */
```

`int a[3][4]` 的元素类型是 `int[4]`，第一维降掉后剩下的是"指向 `int[4]` 的指针"，即 `int (*)[4]`，第二维纹丝不动。这个类型在篇 2 用四要素拆过：`p` 所指向的类型是 `int[4]`，按篇 3 的步长铁律，`p + 1` 跨 16 字节，恰好落在下一行开头。

标准为什么只降第一维？看地址计算就懂了。`a[i][j]` 的地址是：

```
base + i × 4 × sizeof(int) + j × sizeof(int)
base + i × 16 + j × 4
```

第一维长度 3 根本没参与计算，`i` 走到 2 还是 99 都是同一个公式，所以形参里的 3 丢了无所谓；列数 4 却在乘法里站着岗，丢了它地址就算不出来。这不是实现偷懒，是数学上的必需：连续布局下行地址由列宽算出，列宽必须编译期可见。退化"不递归"，是在"类型随传参降维"与"地址计算需要列宽"之间唯一自洽的选择。



图：退化只降第一维。列数在地址公式里站岗，必须保留；三种签名经调整后同为 `int (*)[4]`，编译器眼里没有区别。

三、三种签名，编译器眼里同一个类型

```c
void f1(int (*p)[4], int rows);
void f2(int p[][4],    int rows);
void f3(int p[3][4],   int rows);
```

三种写法经 §6.7.6.3 调整后完全同型，函数原型兼容，可以随意混用。`p[3][4]` 里的 3 写不写、写多少都不影响：它已被调整规则丢弃，留下的唯一作用是给读者标注"预期几行"。真正必须写对的是 4。

两个连带的 `sizeof` 考点也随之定案。函数内 `sizeof(p)` 是 8（它是个指针）；`sizeof(*p)` 是 16（它指向 `int[4]`）。而 `main` 里 `sizeof(a)` 是 48——篇 1 的"身份二"在场，数组名没退化，量的还是整块。

再补一个赋值方向的事实：`int (*p)[4] = a;` 合法，因为 `a` 退化后恰好就是 `int (*)[4]`；而 `&a` 的类型是 `int (*)[3][4]`，指向整个二维数组，步长 48。`a`、`&a`、`&a[0]` 三个地址数值相同，类型其实只有两种：`a` 退化产物与 `&a[0]` 同为 `int (*)[4]`，`&a` 独立一档——篇 1 的"取地址不退化"在这里再次兑现。

四、int** 什么时候对：指针数组是另一种布局

`int **` 并非只有错题。它收的是"装地址的数组"：

```c
int *rows[3];
for (int i = 0; i < 3; i++)
    rows[i] = malloc(4 * sizeof(int));          /* 每行单独一块 */
rows[1] = realloc(rows[1], 8 * sizeof(int));    /* 第二行可以更长 */

int **p = rows;   /* 合法：rows 退化为 int** */
p[2][0] = 9;      /* 正常工作 */
```

`rows` 是指针数组，头 24 字节真有三个地址格，`rows[i]` 读出的是实打实的行地址。这世界与二维数组互斥：`int a[3][4]` 是一块 48 字节连片，数据相邻；指针数组是"3 个地址 + 3 块独立内存"，行与行可以相距十万八千里，还能各行不等长（锯齿数组）。两种布局没法靠强转互换：连片里根本没有地址格可读。

工程上怎么选？连续布局对 CPU 缓存友好，整块一次搬进缓存行，遍历快，但行列必须编译期定死；锯齿布局按需分配、每行可变长，代价是 3 次分配、24 字节开销，以及遍历时的指针追逐。内核与标准库的惯例是：二级指针形参默认假定你给的是指针数组，`argv` 就是样本（篇 3 拆过它的两层图）。给错了布局，类型系统在强转面前沉默，运行时替它说话。

列数运行期才知道、又想要连续布局怎么办？C99 给了正路：变长形参。

```
void print2d(int rows, int cols, int a[rows][cols]) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            printf("%d ", a[i][j]);
}
/* 列宽来自实参，编译器照样能算地址；连续布局与动态列宽兼得 */
```

形参 `a[rows][cols]` 调整为指向"每行 cols 个 int"的指针，这是"运行期列宽 + 连续内存"的官方组合，GCC 手册与不少数值代码里都能见到。

写在最后

1. 二维数组是"连片数据"：48 字节没有一个字节是地址；`int**` 的世界开头必须是地址格，两者不可强转互换。
2. 退化只降第一维（C11 §6.7.6.3）：`int a[3][4]` 传参即 `int (*)[4]`；列数参与地址计算必须保留，第一维没参与所以可丢。
3. `int (*p)[4]`、`int p[][4]`、`int p[3][4]` 三种签名同型；第一维写不写只影响读者，列数写错就是另一个类型。
4. `int**` 收指针数组（argv 模型）：地址格真实存在，支持锯齿行；与二维数组是两种内存布局，不是两种写法。
5. 运行期列宽走 C99 变长形参 `int a[rows][cols]`，连续布局与动态列宽可以兼得。

至此，篇 1 的退化、篇 2 的四要素、篇 3 的步长在这一个形参上完成了会师。下一篇离开类型系统，进标准库的发动机舱：《memcpy 为什么不能拷自己：restrict 的秘密》，`memcpy` 与 `memmove` 的一字之差，藏着 C 标准给优化器发的许可证。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。